

claude-windows / 05c32f87-3bfb-4a7f-861c-0050ac4169ac

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\05c32f87-3bfb-4a7f-861c-0050ac4169ac\subagents\workflows\wf_a8f43370-4cb\agent-a714804389c3a5e06.jsonl`
- SHA-256: `1dbe2835af89695f1470a0bfc70e96fa991866683ae5696ba5be14131a87a122`
- Source modified: `2026-07-07T15:21:38+00:00`
- Imported at: `2026-07-09T08:32:24+00:00`
- project: `wf_a8f43370-4cb`
- session_id: `05c32f87-3bfb-4a7f-861c-0050ac4169ac`
```

Transcript

1. user (2026-07-07T15:13:07.031Z)

You are reviewing a two-repo bug fix BEFORE it merges into a HOSTED PRODUCTION Cloud Run control-plane.

THE BUG: On the hosted control-plane, POST /api/process with compute="local" ran the in-process segmenter ON THE CONTROL-PLANE, which bakes NO detector weights (only the worker Cloud Run Job GCSFuse-mounts weights at /gcs/models/...). It died deep with a cryptic "WASB weights not found at /opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar".
[COMPUTE] requested=local actual=in_process dispatched=None status=failed.

THE FIX (2 parts):

```
- BACKEND (repo undefined, branch undefined, PR #undefined):
  backend/orchestration.py adds _WEIGHTS_BACKED_SEGMENTERS = {wasb_hybrid, tracknet_hybrid,
  fusion_hybrid} and _local_detector_weights_present(cfg, seg_name). In _maybe_dispatch_remote
  the
  choice=="local" branch now fails fast via _failed(msg, False) when the segmenter is weights-
  backed
  AND _cloud_available(cfg) AND local weights are not readable; otherwise returns None (in-
  process as
  before). Plus 4 tests in tests/test_api_cloud_dispatch.py.
- SPA (repo undefined, branch undefined, PR #undefined):
  web/src/components/IngestPanel.tsx compute picker defaults to "cloud" when
  capabilities.deployment.compute_target=="cloud_run" (else keeps the free "local" default).
  Plus 1 test.
```

AUTHORITATIVE CODE ? the working trees may NOT contain the change; read these snapshot files:

```
- undefined/backend.diff (backend commit diff)
- undefined/orchestration_new.py (FULL post-change backend/orchestration.py)
- undefined/test_api_cloud_dispatch_new.py (FULL post-change tests)
- undefined/spa.diff (SPA commit diff)
- undefined/IngestPanel_new.tsx (FULL post-change IngestPanel.tsx)
```

You MAY read other repo files for ground truth, e.g. under undefined:

```
  backend/pipeline/detectors/native_wasb_runner.py, wasb_runner.py, tracknet_runner.py;
  backend/pipeline/segmenters/{wasb_hybrid,tracknet_hybrid,fusion_hybrid,trajectory_hybrid}.py;
  backend/config/models.py; backend/main.py (the /api/capabilities handler ~line 364-410);
  and under undefined: web/src/api/types.ts, web/src/api/client.ts,
  web/src/components/IngestPanel.test.tsx.
```

You may also run: `gh pr diff undefined -R Khelsutra/badminton-highlight-indexer` and `gh pr diff undefined -R avidullu/khelsutra`.

Report ONLY real, concrete defects that would cause wrong behavior, a bad/incorrect merge, a regression, or a materially worse UX ? each with a SPECIFIC failure scenario (concrete config/inputs

? the wrong result). Verify claims against the actual code before reporting; do not invent style nits.

If nothing real exists in your dimension, return an empty findings array.

Adversarially VERIFY this finding from the 'test-adequacy' review. Your job is to REFUTE it if at all possible: read the ACTUAL authoritative code (the snapshot files above and any repo files needed) and determine whether the described failure REALLY occurs against the real code as written. Default to REFUTED unless you can reproduce it concretely. Beware findings that assume a code path that does not exist, or ignore an earlier guard.

FINDING:

- severity: medium
- category: test-coverage
- location: tests/test_api_cloud_dispatch.py (new guardrail block, snapshot lines ~731-829)
- claim: The guardrail is declared for three weights-backed detectors (wasb_hybrid, tracknet_hybrid, fusion_hybrid) but the four new tests only exercise wasb_hybrid and motion; tracknet_hybrid and fusion_hybrid ? and thus the entire branchy half of _local_detector_weights_present (the tracknet weights_path resolution and the fusion substrate switch) ? have ZERO coverage (neither fail-fast nor escape-hatch).
- failure_scenario: A deployment runs segmenter=fusion_hybrid with indexing.fusion.substrate='tracknet' (or segmenter=tracknet_hybrid). A later refactor breaks the substrate routing in _local_detector_weights_present (orchestration.py:179-186) ? e.g. it reads indexing.wasb.weights_path for all three, or TrackNetConfig.weights_path (models.py:51) is renamed. Every new test still passes (they only use wasb_hybrid/motion), yet on the hosted cloud_run control-plane the guardrail now probes the WRONG weights file: WasbIndexConfig.weights_path defaults to a real path (~/models/WASB-SBDT/...), so if that resolves 'present' the escape hatch is wrongly taken and the in-process run falls through and dies deep with the exact 'WASB weights not found' cryptic failure this fix exists to eliminate ? silently, with a green suite.
- suggested_fix: Add at least two tests mirroring test_local_on_cloud_server_without_weights_fails_fast / _with_weights_present_runs_in_process but for (a) segmenter='tracknet_hybrid' driving indexing.tracknet.weights_path, and (b) segmenter='fusion_hybrid' with indexing.fusion.substrate='tracknet' set (tracknet weights missing ? fail-fast; tracknet weights present ? in_process) plus a fusion default-substrate('wasb') case. This pins the substrate switch and the tracknet config path so a routing regression can't ship green.

Return CONFIRMED only with a concrete repro grounded in the real code; else REFUTED with the reason.

2. assistant (2026-07-07T15:13:23.822Z)

I'll start by locating the authoritative snapshot files and the relevant repo files.

3. user (2026-07-07T15:13:26.438Z)

```
total 108
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:24 ..
drwxr-xr-x 1 avidu 197609    0 Jun 23 09:53 Annotation Setup
-rw-r--r-- 1 avidu 197609 905 Jul  2 21:20 AUDIT_REPORT_khelsutra-guru_2026-07-02.md
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 badminton-highlight-indexer
-rw-r--r-- 1 avidu 197609 1222 Jul  2 21:35 bash.exe.stackdump
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:34 bhi-wt-worker-reval
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 khelsutra
drwxr-xr-x 1 avidu 197609    0 Jul  5 18:47 khelsutra-screencast
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:44 marketing
drwxr-xr-x 1 avidu 197609    0 Jul  5 13:43 pr-158-screenshots
drwxr-xr-x 1 avidu 197609    0 Jun 22 15:03 rally-annotator
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 sports-data-collector
drwxr-xr-x 1 avidu 197609    0 Jul  2 19:46 sports-obsreport
---KHELSUTRA---
total 62
drwxr-xr-x 1 avidu 197609    0 Jul  5 12:57 .
```

```
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 22 11:12 .claude
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:38 .git
-rw-r--r-- 1 avidu 197609 297 Jun 20 23:27 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jun 20 12:43 .github
-rw-r--r-- 1 avidu 197609 323 Jun 22 14:57 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  2 21:13 docs
drwxr-xr-x 1 avidu 197609    0 Jun 21 23:55 locales
-rw-r--r-- 1 avidu 197609 9773 Jul  2 21:13 NEXT_STEPS.md
drwxr-xr-x 1 avidu 197609    0 Jun 21 15:38 node_modules
-rw-r--r-- 1 avidu 197609 1469 Jun 23 00:19 README.md
drwxr-xr-x 1 avidu 197609    0 Jun 23 00:19 site
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:45 web
```

4. user (2026-07-07T15:13:29.825Z)

(Bash completed with no output)

5. assistant (2026-07-07T15:13:34.210Z)

The snapshot files use placeholder paths. Let me find the actual repo and snapshot files.

6. user (2026-07-07T15:13:40.298Z)

```
total 742
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 .
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:57 ..
drwxr-xr-x 1 avidu 197609    0 Jun 25 23:31 .claude
-rw-r--r-- 1 avidu 197609 94208 Jul  7 20:14 .coverage
-rw-r--r-- 1 avidu 197609  611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609   69 Jun 10 23:25 .env
-rw-r--r-- 1 avidu 197609 1059 Jul  7 18:29 .gcloudignore
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:43 .git
-rw-r--r-- 1 avidu 197609  947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609 1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609    0 Jul  7 10:27 __pycache__
drwxr-xr-x 1 avidu 197609    0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609    0 Jul  7 19:56 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609  6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609  2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609    0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:43 docs
drwxr-xr-x 1 avidu 197609    0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609  1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609    0 Jul  7 20:14 output
-rw-r--r-- 1 avidu 197609  5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609   240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 rally_index.db
-rw-r--r-- 1 avidu 197609 35511 Jul  2 08:13 README.md
-rw-r--r-- 1 avidu 197609  1674 Jul  2 20:05 requirements.txt
-rw-r--r-- 1 avidu 197609   585 Jul  2 19:49 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609   809 Jun 25 16:24 ruff.toml
-rwxr-xr-x 1 avidu 197609  2118 Jun 29 13:47 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5739 Jun 29 13:47 run_eval.py
-rw-r--r-- 1 avidu 197609  4594 Jun 29 13:47 run_regression.py
-rwxr-xr-x 1 avidu 197609  5073 Jul  7 10:27 score_golden_gate.py
```

```

-rwxr-xr-x 1 avidu 197609 7617 Jul 7 03:32 score_local.py
---bhi-wt-worker-reval---
total 362
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:34 .
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:57 ..
-rw-r--r-- 2 avidu 197609 94208 Jul 7 20:34 .coverage
-rw-r--r-- 1 avidu 197609 611 Jul 7 19:57 .coveragerc
-rw-r--r-- 1 avidu 197609 1059 Jul 7 19:57 .gcloudignore
-rw-r--r-- 1 avidu 197609 110 Jul 7 19:57 .git
-rw-r--r-- 1 avidu 197609 947 Jul 7 19:57 .gitattributes
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:57 .github
-rw-r--r-- 1 avidu 197609 1998 Jul 7 19:57 .gitignore
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:08 .mypy_cache
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:06 .pytest_cache
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:06 .ruff_cache
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:57 .vscode
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:09 __pycache__
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:05 backend
-rw-r--r-- 1 avidu 197609 6619 Jul 7 19:57 CLAUDE.md
-rw-r--r-- 1 avidu 197609 2735 Jul 7 19:57 config.json
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:57 deploy
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:57 docs
drwxr-xr-x 1 avidu 197609 0 Jul 7 19:57 eval_baselines
-rw-r--r-- 1 avidu 197609 1999 Jul 7 19:57 mypy.ini
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:34 output
-rw-r--r-- 1 avidu 197609 5413 Jul 7 19:57 pyproject.toml
-rw-r--r-- 1 avidu 197609 245 Jul 7 19:57 pytest.ini
-rw-r--r-- 1 avidu 197609 35511 Jul 7 19:57 README.md
-rw-r--r-- 1 avidu 197609 1674 Jul 7 19:57 requirements.txt
-rw-r--r-- 1 avidu 197609 585 Jul 7 19:57 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609 809 Jul 7 19:57 ruff.toml
-rwxr-xr-x 1 avidu 197609 2118 Jul 7 19:57 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5739 Jul 7 19:57 run_eval.py
-rw-r--r-- 1 avidu 197609 4594 Jul 7 19:57 run_regression.py
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:09 scripts
-rw-r--r-- 1 avidu 197609 2410 Jul 7 19:57 SESSION_HANDOFF.md
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:28 tests
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:09 tools
drwxr-xr-x 1 avidu 197609 0 Jul 7 20:09 training

```

7. user (2026-07-07T15:13:41.157Z)

```

---diffs---
---orchestration.py real---
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer/backend/orchestration.py
C:\Users\avidu\Projects\khelsutra-guru\bhi-wt-worker-reval/backend/orchestration.py

```

8. assistant (2026-07-07T15:13:49.402Z)

Snapshot files with `undefined/` prefix don't exist literally. I need to work with the actual repos. There are two: `badminton-highlight-indexer` and a worktree `bhi-wt-worker-reval`. Let me check git state and find the changed code.

9. user (2026-07-07T15:13:53.241Z)

```

=== main repo branch ===
fix/cloud-run-exec-status-fastfail
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)

=== worktree branch ===
fix/worker-skip-revalidation-on-dispatch

```

8586c34 fix(serving): worker skips redundant re-validation on the cloud dispatch path
3be0935 docs(runlog): DEPLOY_REPORT addendum ? control-plane redeployed to the forwarding image (#510)
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
143dbb9 feat(serving): default cloud-serving to nvdec_resident decode (#508)
44a2f47 feat(detectors): GPU-resident WASB NVDEC decode behind default-OFF decode_backend flag (#507)

10. user (2026-07-07T15:14:01.706Z)

Found 15 files

C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\tests\test_api_cloud_dispatch.py
C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\backend\orchestration.py
C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\backend\worker__main__.py
C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\tests\test_main_module_budget.py
C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\docs\archives\past_projects\AUDIT_REPORT_kheIsutra-guru_2026-07-02-COMPLETED-2026-07-02.md
C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\backend\main.py
C:\Users\avidu\Projects\kheIsutra-guru\bhi-wt-worker-reval\backend\api\job_worker.py
C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer\tests\test_api_cloud_dispatch.py
C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer\backend\orchestration.py
C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer\backend\main.py
C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer\backend\api\job_worker.py
C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer\tests\test_main_module_budget.py
C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer\docs\COMPUTE_DECOUPLED_SERVING\runlogs\DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
C:\Users\avidu\Projects\kheIsutra-guru\badminton-highlight-indexer\docs\archives\past_projects\AUDIT_REPORT_kheIsutra-guru_2026-07-02-COMPLETED-2026-07-02.md

11. assistant (2026-07-07T15:14:07.323Z)

Both repos already contain the change. The main repo is on branch `fix/cloud-run-exec-status-fastfail`. Let me read the authoritative code: the orchestration functions and the tests.

12. user (2026-07-07T15:14:09.030Z)

```
391: def _maybe_dispatch_remote(
```

13. assistant (2026-07-07T15:14:15.392Z)

Interesting ? the main repo only matched `\_maybe\_dispatch\_remote`, not `\_WEIGHTS\_BACKED\_SEGMENTERS`. The change may only be in the worktree. Let me check both precisely.

14. user (2026-07-07T15:14:17.063Z)

No matches found

15. user (2026-07-07T15:14:20.692Z)

```
=== main repo: does orchestration have the fix? ===  
0  
=== git status main ===  
M backend/compute/__init__.py  
M backend/compute/base.py  
M backend/compute/cloud_run.py  
M backend/worker/__main__.py  
M tests/test_compute_backend.py  
?? score_golden_gate.py
```

```
?? score_local.py
=== git log for orchestration fix ===
66756ad chore(serving): forward WASB decode/tuning knobs to worker + release hardening (#509)
f6864f1 feat(recovery): restart-safe remote jobs + read-path re-hydrate (CP-rebuild P3, #471
#485) (#494)
4cd36fd feat(progress): live cloud-poll heartbeat ? job.progress moves every tick during a GPU
run (CP-rebuild P6 slice, #482) (#493)
851f21d feat(process): submit-time MD5 + completed-work cache-reuse ? same video never re-runs
the GPU (CP-rebuild P4, #484) (#492)
63d65f9 fix(cloud-serving): serve large reels via a gs:// signed URL (#463) (#469)
ffe68ed fix(cloud-serving): parallelize the drain under cloud_run (#466) (#468)
d7ebe9c fix(cloud-serving): stage browser-uploaded local files to gs:// before L4 dispatch
(#461) (#467)
e4914d1 Surface validation failure reasons
```

16. assistant (2026-07-07T15:14:28.766Z)

Neither working tree contains the fix ? confirming the task note. The authoritative snapshot files must be written somewhere by the orchestrator. Let me search broadly for them.

17. user (2026-07-07T15:14:35.171Z)

```
C:\Users\avidu\AppData\Local\Temp\backend.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/backend.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/IngestPanel_new.tsx
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/spa.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/test_api_cloud_dispatch_new.py
C:\Users\avidu\AppData\Local\Temp\spa.diff
```

18. user (2026-07-07T15:16:16.746Z)

```
C:\Users\avidu/.cache/codex-runtimes/codex-primary-runtime/plugins/openai-primary-
runtime/plugins/presentations/skills/presentations/assets/builtin_templates/codex-grid-layout-
library/assets/previews
C:\Users\avidu/.claude/plugins/marketplaces/claude-plugins-official/plugins/code-review
C:\Users\avidu/.claude/plugins/marketplaces/claude-plugins-official/plugins/pr-review-toolkit
C:\Users\avidu/.codex/.tmp/plugins/plugins/coderabbit/skills/coderabbit-review
C:\Users\avidu/.codex/.tmp/plugins/plugins/daloopa/skills/earnings-review
C:\Users\avidu/.codex/.tmp/plugins/plugins/domotz-preview
C:\Users\avidu/.codex/.tmp/plugins/plugins/shopify/skills/shopify-app-store-review
C:\Users\avidu/.codex/.tmp/plugins/plugins/superpowers/skills/receiving-code-review
C:\Users\avidu/.codex/.tmp/plugins/plugins/superpowers/skills/requesting-code-review
C:\Users\avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills/accessibility-review
--- broad new.py search ---
C:\Users\avidu\AppData\Local\Temp\backend.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/backend.diff
C:\Users\avidu\AppData\Local\Temp\claude/c--Users-avidu-Projects-badminton-highlight-
indexer/05c32f87-3bfb-4a7f-861c-0050ac4169ac/scratchpad/orchestration_new.py
```

19. assistant (2026-07-07T15:16:21.747Z)

Found the snapshot files in the scratchpad. Let me read the authoritative post-change code.

20. user (2026-07-07T15:16:23.092Z)

```
1      """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
```

```

2
3     The full hash -> validate -> segment -> report pipeline (`_execute_processing`), the
4     Cloud Run dispatch + ingest path (`maybe_dispatch_remote` /
5     `_ingest_remote_segments`),
6     and the shared compile resolution + worker handler (`_resolve_compile` /
7     `_execute_compile`).
8     Extracted verbatim from `backend.main` to shrink the orchestrator; behavior is
9     unchanged.
10
11     SEAM DISCIPLINE (same convention as `backend.api.job_worker`): anything that must
12     remain
13     monkeypatchable-on-`backend.main` or that lives in `backend.main` (the transitional
14     `db_instance`/`global_config` globals and the `ProcessRequest` model) is resolved
15     through
16     the `backend.main` module object at call time (`import backend.main as _main`),
17     never as a
18     bare local. `backend.main` re-exports every name below, so
19     `monkeypatch.setattr(backend.main,
20     ...)` and `import backend.main as m; m._execute_processing(...)` keep working
21     unchanged.
22     This module must NOT import `backend.main` at module level (main imports us for the
23     re-export).
24     Routed seams (rebind-patched on `backend.main` by tests): `global_config`,
25     `db_instance`,
26     `ProcessRequest`, `compute_video_id`, `emit_indexer_report`, `VideoCompiler`.
27     Object-attribute patches (`m.storage.fetch`, `m.segmenter_registry.get`, ...) hit
28     the shared
29     objects and need no routing.
30     """
31
32     import csv
33     import json
34     import logging
35     import math
36     import os
37     import re
38     import tempfile
39     import threading
40     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
41     from urllib.parse import quote
42
43     from backend import storage
44     from backend.config import AppConfig, StitchingConfig
45     from backend.pipeline.segmenters.base import segmenter_registry
46     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
47     from backend.pipeline.validators.base import registry
48     from backend.results import Failure
49     from backend.utils.paths import output_base, safe_output_filename
50
51     if (
52         TYPE_CHECKING
53     ): # runtime import would be circular (main imports us for the re-export)
54         from backend.main import ProcessRequest # noqa: F401
55
56     logger = logging.getLogger(__name__)
57
58     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
59     compile stitch ?
60     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
61     dispatch/poll) never
62     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
63     Gemini is
64     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
65     remote jobs overlap
66     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no

```

```

deadlock; when the
52     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
53     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
54
55
56     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
57         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
58
59         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
60         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
61         and
62         keep the prior good results intact ? a failed/empty re-run never destroys prior
63         data.
64         """
65         if segments:
66             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
67         else:
68             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
69
70     def _validation_failure_message(failures: List[Any]) -> str:
71         details = []
72         for failure in failures:
73             name = getattr(failure, "validator_name", None) or "validation"
74             message = (getattr(failure, "message", None) or "").strip()
75             details.append(f"{name}: {message}" if message else str(name))
76         if not details:
77             return "Video failed validation checks."
78         return "Video failed validation: " + "; ".join(details)
79
80     def _report_config_for_input(config: Any, raw_input: str) -> Any:
81         """Return report config that keeps remote-input reports out of disposable
82         scratch."""
83         storage_config = (
84             config.get("storage")
85             if isinstance(config, dict)
86             else getattr(config, "storage", None)
87         )
88         try:
89             if storage.input_is_local(raw_input, storage_config):
90                 return config
91         except ValueError:
92             return config
93         try:
94             cfg = (
95                 config.model_dump(mode="json")
96                 if hasattr(config, "model_dump")
97                 else dict(config or {})
98             )
99         except Exception:
100             cfg = {}
101         report = dict(cfg.get("report") or {})
102         if report.get("output_dir"):
103             return config
104         report["output_dir"] = os.path.join(output_base(config), "reports")
105         cfg["report"] = report
106         return cfg
107
108     def _ingest_remote_segments(
109         db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
110     ) -> int:
111         """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
112         the bucket and

```



```

112         persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
call ? so
113         ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
were produced.
114         Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
pattern (cloud
115         backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
malformed row is
116         skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
temp dir is
117         cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
a job failure
118         and prunes any partial rows (destroy-AFTER-confirm)."""
119         uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
120         ref = storage.parse_uri(uri)
121         n = 0
122         with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
123             local = storage.fetch(
124                 uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
125             )
126             with open(local, newline="", encoding="utf-8") as f:
127                 for row in csv.DictReader(f):
128                     if n >= max_rows:
129                         logger.warning(
130                             "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
rest.",
131                             max_rows,
132                             uri,
133                         )
134                         break
135                     try:
136                         start, end = float(row["start"]), float(row["end"])
137                     except (KeyError, TypeError, ValueError):
138                         continue
139                     if not (math.isfinite(start) and math.isfinite(end)):
140                         continue # reject inf/nan times rather than persist a junk rally
141                     meta: Dict[str, Any] = {
142                         "source": "cloud_run",
143                         "ending_reason": (row.get("ending_reason") or "").strip(),
144                     }
145                     sc = (row.get("shot_count") or "").strip()
146                     if sc:
147                         try:
148                             meta["shot_count"] = int(
149                                 float(sc)
150                             ) # OverflowError on 'inf'/'1e400'
151                         except (ValueError, OverflowError):
152                             meta["shot_count"] = sc
153                     db.add_play_segment(video_id, start, end, metadata=meta)
154                     n += 1
155         return n
156
157
158     # Trajectory segmenters whose IN-PROCESS detect needs model weights present on THIS box.
A hosted
159     # cloud-serving control-plane bakes NO detector weights into its image (only the worker
Cloud Run Job
160     # GCSFuse-mounts them), so forcing compute="local" for one of these there can never
succeed ? the
161     # guardrail in _maybe_dispatch_remote fails such a request FAST instead of the cryptic
deep
162     # "WASB weights not found". (Names are the registered segmenter identities; see
backend/pipeline/segmenters.)
163     _WEIGHTS_BACKED_SEGMENTERS = frozenset({"wasb_hybrid", "tracknet_hybrid",
"fusion_hybrid"})

```

```

164
165
166     def _local_detector_weights_present(cfg, seg_name: str) -> bool:
167         """Whether the trajectory weights the IN-PROCESS ``seg_name`` detector needs are
present + readable
168         on THIS box. Mirrors the native/WSL runners' resolution (env override ?
``indexing.<det>.weights_path``)
169         and the ``/api/capabilities`` probe, so the guardrail agrees with the healthcheck
that would
170         otherwise fail deep in the run. Fusion follows its configured substrate (WASB
default / TrackNet).
171         A non-weights segmenter returns True (nothing local to check)."""
172         name = (seg_name or "").strip().lower()
173         if name not in _WEIGHTS_BACKED_SEGMENTERS:
174             return True
175         idx = getattr(cfg, "indexing", None)
176         substrate = (
177             getattr(getattr(idx, "fusion", None), "substrate", "") or "wasb"
178         ).strip().lower()
179         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180             path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181         else: # wasb_hybrid, or fusion on the (default) WASB substrate
182             path = (
183                 os.environ.get("WASB_WEIGHTS")
184                 or getattr(getattr(idx, "wasb", None), "weights_path", "")
185                 or ""
186             )
187         return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190     def _cloud_available(cfg) -> bool:
191         """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
3).
192
193         True when the control plane is wired for Cloud Run: either the configured default
target is
194         already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
coordinates
195         (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
(``storage.output_root``)
196         are present so a forced per-request override can actually dispatch + collect a
result.
197
198         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work."""
199         deployment = getattr(cfg, "deployment", None)
200         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201             return True
202         has_job = bool(
203             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204         )
205         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206         return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213         (queue/cold-start), so the very first tick already shows movement."""
214         minutes = int(elapsed_sec) // 60
215         if minutes < 1:

```

```

216         return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217     return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222     object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223     deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224     ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
225     if getattr(input_ref, "backend", "") != "gcs":
226         return None
227     sc = _storage_settings(cfg)
228     if not sc:
229         return None
230     try:
231         st = storage.get_storage("gcs", config=sc).stat(input_ref)
232         return getattr(st, "md5", None) or None
233     except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234         logger.info(
235             "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236             getattr(input_ref, "uri", input_ref),
237             exc_info=True,
238         )
239     return None
240
241
242 def _cached_process_payload(
243     video_id: str, validation_results, segments, requested_compute
244 ) -> Dict[str, Any]:
245     """The /api/process result for a video whose detection already exists ? ONE shape
for
246     both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247     return {
248         "video_id": video_id,
249         "status": "success",
250         "message": (
251             f"Reused {len(segments)} cached play segments. "
252             "Pass force=true to reprocess this video."
253         ),
254         "results": validation_results or [],
255         "play_segments": segments,
256         "cached": True,
257         "compute": {"path": "cached", "requested": requested_compute},
258     }
259
260
261 def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262     """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263     ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
264     ref = storage.parse_uri(uri)
265     with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
266         local = storage.fetch(
267             uri, os.path.join(td, ref.name or "report.json"), config=storage_config
268         )
269         with open(local, encoding="utf-8") as f:
270             loaded = json.load(f)
271         return loaded if isinstance(loaded, dict) else None
272
273
274 # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache

```

```

hits
275     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
276     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
277     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
278     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
279     _REHYDRATE_LOCKS_GUARD = threading.Lock()
280
281
282     def _rehydrate_lock(video_id: str) -> threading.Lock:
283         with _REHYDRATE_LOCKS_GUARD:
284             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
285
286
287     def probe_process_cache(db, cfg, video_id: str) -> bool:
288         """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
289
290         Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
291         ordering). Used to decide whether a submit can skip the local-scratch fetch guard
292         (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
293         errors report False (? the normal dispatch path with its guards)."""
294         try:
295             existing = db.get_video(video_id)
296             if (
297                 existing
298                 and existing.get("status") == "processed"
299                 and db.query_play_segments(video_id, {})
300             ):
301                 return True
302             sc = _storage_settings(cfg)
303             if not sc or (sc.get("backend") or "local") == "local":
304                 return False
305             output_root = (sc.get("output_root") or "").rstrip("/")
306             if not output_root:
307                 return False
308             report_uri = f"{output_root}/outputs/{video_id}/report.json"
309             if not storage.exists(report_uri, config=sc):
310                 return False
311             report = _read_bucket_json(report_uri, sc)
312             return bool(report and report.get("status") == "success")
313         except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
314             logger.warning(
315                 "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
316             )
317             return False
318
319
320     def cached_process_result(
321         db, cfg, video_id: str, video_path: str, requested_compute=None
322     ) -> Optional[Dict[str, Any]]:
323         """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324         the DB first, then the bucket (``outputs/<md5>/report.json`` with ``status=success``
325         re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
326         source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
327         completed work found (or the check failed) ? the caller dispatches normally. The
bucket
328         branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),
so a
329         partial earlier ingest is replaced, never double-counted.

```

```

330
331 Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
332 per-`video_id` lock ? a concurrent caller blocks, then finds the winner's rows via
333 the
334 in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
335 job before invoking (writes happen post-claim)."""
336 try:
337     with _rehydrate_lock(video_id):
338         existing = db.get_video(video_id)
339         if existing and existing.get("status") == "processed":
340             segs = db.query_play_segments(video_id, {})
341             if segs:
342                 return _cached_process_payload(
343                     video_id,
344                     existing.get("validation_results", []),
345                     segs,
346                     requested_compute,
347                 )
348             sc = _storage_settings(cfg)
349             if not sc or (sc.get("backend") or "local") == "local":
350                 return None
351             output_root = (sc.get("output_root") or "").rstrip("/")
352             if not output_root:
353                 return None
354             outputs_uri = f"{output_root}/outputs/{video_id}"
355             report_uri = f"{outputs_uri}/report.json"
356             if not storage.exists(report_uri, config=sc):
357                 return None
358             report = _read_bucket_json(report_uri, sc)
359             if not report or report.get("status") != "success":
360                 return None
361             if not video_path:
362                 # Read-path re-hydrate (#485): a wiped DB has no source record. The
363                 # carries the worker's --video-uri (P3); older reports fall back to the
364                 # staged-canonical videos/<md5> prefix; else "" ? the segments still
365                 # recover, and the next same-md5 submit re-binds the truthful path via
366                 # add_video's upsert.
367                 video_path = str(report.get("video_uri") or "") or _first_object_under(
368                     f"{output_root}/videos/{video_id}/", sc
369                 )
370             play_wm, cand_wm = db.segment_watermarks(video_id)
371             db.add_video(video_id, video_path, "processed", [])
372             n = _ingest_remote_segments(db, video_id, outputs_uri, sc)
373             if n <= 0:
374                 return None
375             segments = [
376                 s
377                 for s in db.query_play_segments(video_id, {})
378                 if int(s.get("id", 0)) > play_wm
379             ]
380             _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
381             if not segments:
382                 return None
383             logger.info(
384                 "[API] re-hydrated %d segments for video_id=%s from %s (submit-time
385                 cache hit)",
386                 n,
387                 video_id,
388                 outputs_uri,
389             )
390             return _cached_process_payload(video_id, [], segments, requested_compute)
391 except Exception: # noqa: BLE001 ? cache misses must degrade to a normal dispatch
392     logger.warning(

```

```

391         "[API] submit-time cache check failed for %s ? dispatching normally",
392         video_id,
393         exc_info=True,
394     )
395     return None
396
397
398     def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
399         """The first object URI under a bucket prefix, or "" ? best-effort source
recovery."""
400         try:
401             return next(iter(storage.list_uris(prefix_uri, config=storage_config)), "")
402         except Exception: # noqa: BLE001 ? recovery nicety, never a failure
403             return ""
404
405
406     _MD5_HEX_RE = re.compile(r"[0-9a-f]{32}")
407
408
409     def read_path_rehydrate(db, cfg, video_id: str) -> bool:
410         """#485, the read half of "DB = cache, bucket = truth": before a rally READ reports
411         nothing for an md5-shaped id, try re-hydrating the video from ``outputs/<md5>/` ` ?
so
412         ``?video=<md5>`` lands on grounded state after any restart/redeploy wipe. True ? the
DB
413         has the video's rows now (re-query). Non-md5 ids return False without touching the
414         bucket; every failure degrades to False (the read just reports what the DB has)."""
415         vid = (video_id or "").strip().lower()
416         if not _MD5_HEX_RE.fullmatch(vid):
417             return False
418         if not probe_process_cache(db, cfg, vid):
419             return False
420         return cached_process_result(db, cfg, vid, "") is not None
421
422
423     def _maybe_dispatch_remote(
424         cfg,
425         db,
426         req: "ProcessRequest",
427         video_id: str,
428         seg_name: str,
429         val_results,
430         play_wm: int,
431         cand_wm: int,
432         notify,
433     ) -> Optional[Tuple[Dict[str, Any], bool]]:
434         """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
``deployment.compute_target
435         == "cloud_run``, run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
intervals into
436         THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
437
438         Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
the cloud job
439         actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
observability report
440         doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
process segmenter
441         (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
non-``cloud_run`` target, so
442         the in-process path stays byte-identical)."""
443         from backend.compute import ComputeJobSpec, get_compute_backend
444
445         def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
446             # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows

```

```

(id > play_wm);
447         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
448         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
449         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
450         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
451         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
452         # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
453         return (
454             {
455                 "video_id": video_id,
456                 "status": "failed",
457                 "message": msg,
458                 "results": [r.model_dump() for r in val_results],
459                 "play_segments": [],
460                 "compute": {
461                     "path": "cloud_run" if ran else "not_run",
462                     "requested": req.compute,
463                     "target": "cloud_run",
464                     "dispatched": ran,
465                 },
466             },
467             ran,
468         )
469
470         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
471         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
472         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
473         choice = (req.compute or "").strip().lower()
474         if choice and choice not in ("local", "cloud"):
475             logger.warning(
476                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
477                 req.compute,
478             )
479             choice = ""
480         if choice == "local":
481             # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482             # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483             # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484             # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485             # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486             # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487             # below exactly as before (the weights-present check is the escape hatch).
488             if (
489                 (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490                 and _cloud_available(cfg)
491                 and not _local_detector_weights_present(cfg, seg_name)
492             ):
493                 return _failed(
494                     f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                     "live only on the cloud worker, not on the control-plane. Choose 'Run in

```

```

cloud'.",
496         False,
497     )
498     return None # explicit "run on my machine" ? in-process regardless of the
server's target
499     if choice == "cloud":
500         if not _cloud_available(cfg):
501             return _failed(
502                 "cloud-serving was requested but this server isn't configured for it "
503                 "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                 False,
505             )
506         compute = get_compute_backend(cfg, target_override="cloud_run")
507     else:
508         compute = get_compute_backend(
509             cfg
510         ) # server default (default-OFF unless cloud_run)
511     if compute is None:
512         return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514     storage_cfg = getattr(cfg, "storage", None)
515
516     # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517     try:
518         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519     except ValueError as e:
520         return _failed(
521             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522         )
523     if input_ref.backend == "local":
524         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525         # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526         # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.
527         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530         # default, while a bare/local-looking root can't stage no matter what
input_backend says
531         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532         # gives us nothing to build the staged URI under).
533         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534         try:
535             root_backend = (
536                 storage.parse_uri(input_root).backend if input_root else "local"
537             )
538         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539             root_backend = "local"
540         if root_backend == "local":
541             return _failed(
542                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                 "Run job.",
545                 False,

```



```

546         )
547         local_src = input_ref.key # resolved local path of the upload
548         staged_uri =
549         f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
550         try:
551             notify("staging upload to bucket")
552             # Thread the active storage settings so backend-specific config survives (S3
553             endpoint_url/
554             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
555             default client.
556             storage.put(
557                 local_src,
558                 staged_uri,
559                 config=(
560                     storage_cfg.model_dump() if storage_cfg is not None else None
561                 ),
562             )
563         except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
564             failure, not a 500
565             logger.error(
566                 "[API] cloud-serving: failed to stage %s -> %s: %s",
567                 local_src,
568                 staged_uri,
569                 e,
570                 exc_info=True,
571             )
572             return _failed(
573                 f"cloud-serving: failed to stage the uploaded file to {input_root}:
574                 {e}",
575                 False,
576             )
577             # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
578             the L4 job
579             # DETECTs from it, and a later /api/compile re-fetches the same source (the
580             local upload is
581             # gone once this instance recycles). add_video is an UPSERT ? it only updates
582             filepath.
583             req.video_path = staged_uri
584             input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
585             db.add_video(
586                 video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
587             )
588             out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
589             if not out_root:
590                 return _failed(
591                     "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
592                     bucket) "
593                     "? that's where the Cloud Run job writes the result.",
594                     False,
595                 )
596             notify("dispatching (cloud_run)")
597             logger.info(
598                 "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
599                 req.video_path,
600                 out_root,
601             )
602             # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
603             with
604             # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
605             needed); without
606             # forwarding the cloud job would default to the handoff and yield 0 rallies when it
607             has no key.
608             # (player_pool / max_frames are still dropped on the cloud path ? a separate
609             documented follow-up.)

```

```

598     overrides = []
599     idx_cfg = getattr(cfg, "indexing", None)
600     sk = getattr(idx_cfg, "skip_ai_handoff", None)
601     if sk is not None:
602         overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603     # Forward the resolved WASB serving knobs so the PRESET governs the worker
604     (#506/#507). The
605     # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
606     preset itself ?
607     # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
608     model
609     # defaults (cpu, batch 8) regardless of what the control-plane resolved.
610     decode_backend is
611     # cuda-only (validator-enforced) and requires a worker image that understands
612     --decode-backend
613     # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
614     parity default).
615     wasb_cfg = getattr(idx_cfg, "wasb", None)
616     if wasb_cfg is not None:
617         decode_backend = getattr(wasb_cfg, "decode_backend", None)
618         if decode_backend:
619             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
620         batch_size = getattr(wasb_cfg, "batch_size", None)
621         if batch_size:
622             overrides.append(f"indexing.wasb.batch_size={batch_size}")
623         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
624         if prefetch:
625             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
626     spec = ComputeJobSpec(
627         video_uri=req.video_path,
628         out_uri=out_root,
629         segmenter=seg_name,
630         config_overrides=tuple(overrides),
631     )
632     try:
633         # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
634         # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
635         # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
636         which
637         # also gives the SPA a stall signal to time out on, instead of a blind wall
638         clock.
639         result = compute.run_infer(
640             spec,
641             on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
642         )
643     except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
644         failure, not a 500
645         logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
646         return _failed(f"cloud dispatch error: {e}", True)
647     if result.returncode != 0:
648         return _failed(f"cloud job failed (returncode {result.returncode}).", True)
649     if result.video_id and result.video_id != video_id:
650         # The worker re-hashes the bucket bytes; a divergence means the input changed
651         between this
652         # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
653         diagnosable.
654         logger.warning(
655             "[API] cloud video_id %s != local %s ? input bytes differ between the API
656             hash "
657             "and the worker hash; ingesting under the local id.",
658             result.video_id,
659             video_id,
660         )
661     notify("ingesting (cloud_run)")

```

```

651     storage_dict = (
652         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
653         if hasattr(storage_cfg, "model_dump")
654         else {}
655     )
656     try:
657         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
658     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
659         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
660         return _failed(
661             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
662             True,
663         )
664
665     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
666     segments = [
667         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
668     ]
669     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
670     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
671     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
672     provenance = {
673         "path": "cloud_run",
674         "requested": req.compute,
675         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
676         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
677         "execution": result.execution,
678         "outputs_uri": result.outputs_uri,
679     }
680     logger.info(
681         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
682         result.execution,
683         provenance["job"],
684         provenance["region"],
685         result.outputs_uri,
686         video_id,
687     )
688     return (
689         {
690             "video_id": video_id,
691             "status": "success",
692             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
693             "results": [r.model_dump() for r in val_results],
694             "play_segments": segments,
695             "compute": provenance,
696         },
697         True,
698     )
699
700
701     def _execute_processing(
702         config=None, db=None, req=None, progress=None
703     ) -> Dict[str, Any]:
704         """The full hash ? validate ? segment ? report pipeline for one video.
705
706         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
707         module-level ``global_config``/``db_instance`` (backward-compatible with existing
708         tests that pass only ``req``). Route handlers always pass explicit params; older

```

```

709     test code that calls ``_execute_processing(req)`` still works.
710
711     ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
712     Raises on unexpected internal errors ? the caller records those as a job failure
713     """
714     import backend.main as _main # call-time seam: main's globals/model stay patchable
715
716     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
717     # shift positional args ? req arrives in position 0 (config), progress in position 1
(db).
718     if config is not None and not isinstance(config, AppConfig):
719         # Old signature: _execute_processing(req, progress=None)
720         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
721             req = db
722             db = None
723         elif req is None and isinstance(config, _main.ProcessRequest):
724             req = config
725             config = None
726     # Resolve config/db from module globals when not passed explicitly
727     if config is None:
728         config = _main.global_config
729     if db is None:
730         db = _main.db_instance
731
732     notify = progress or (lambda stage: None)
733
734     notify("hashing")
735     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
736     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
737     # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
738     # consumers (hash / validators / segmenter) use the resolved local path.
739     local_video = ""
740     try:
741         local_video = storage.acquire_local_input(
742             req.video_path, getattr(config, "storage", None)
743         )
744         video_id = _main.compute_video_id(local_video)
745     except Exception:
746         storage.cleanup_acquired_local_input(
747             req.video_path, local_video, getattr(config, "storage", None)
748         )
749         raise
750     try:
751         existing_video = db.get_video(video_id)
752     except Exception:
753         storage.cleanup_acquired_local_input(
754             req.video_path, local_video, getattr(config, "storage", None)
755         )
756         raise
757     was_reingest = existing_video is not None
758
759     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
760     default_seg = getattr(
761         getattr(config, "indexing", None), "default_segmenter", "motion"
762     )
763     seg_name = req.segmenter or default_seg
764     segmenter_actual = None
765     segmentation_ran = False
766     val_results: List[Any] = []
767     val_context: Dict[str, Any] = {}
768     # Compute-path PROVENANCE (the validation method): which backend actually ran the

```

```

DETECT ?
769         # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
770         # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
771         compute_actual: Optional[str] = None
772         response: Optional[Dict[str, Any]] = None
773         try:
774             if (
775                 existing_video
776                 and existing_video.get("status") == "processed"
777                 and not req.force
778             ):
779                 cached_segments = db.query_play_segments(video_id, {})
780                 if cached_segments:
781                     logger.info(
782                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
783                         video_id,
784                     )
785                 response = _cached_process_payload(
786                     video_id,
787                     existing_video.get("validation_results", []),
788                     cached_segments,
789                     getattr(req, "compute", None),
790                 )
791                 return response
792
793         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
794         notify("validating")
795         logger.info(f"Running validations for {req.video_path}...")
796         val_results, val_context = registry.run_all_with_context(local_video, config)
797         failures = [r for r in val_results if not r.passed]
798
799         if failures:
800             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
801             # status; it no longer destroys the video's prior good segments.
802             db.add_video(
803                 video_id,
804                 req.video_path,
805                 "failed",
806                 [r.model_dump() for r in val_results],
807             )
808             response = {
809                 "video_id": video_id,
810                 "status": "failed",
811                 "message": _validation_failure_message(failures),
812                 "results": [r.model_dump() for r in val_results],
813             }
814             return response
815
816         # 2. Run segmenter & indexer
817         logger.info("[API] Video passed checks. Segmenting and indexing...")
818         db.add_video(
819             video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
820         )
821
822         segmenter_cls = segmenter_registry.get(seg_name)
823         if not segmenter_cls:
824             # Submit-time validation already 400s unknown names; this is the defensive
825             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
826             available = list(segmenter_registry.list_available().keys())
827             response = {

```

```

828         "video_id": video_id,
829         "status": "failed",
830         "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
831         "results": [r.model_dump() for r in val_results],
832         "play_segments": [],
833     }
834     return response
835
836     logger.info(f"[API] Using segmenter: {seg_name}")
837     notify(f"segmenting ({seg_name})")
838     segmenter_actual = seg_name
839     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
840     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
841     play_wm, cand_wm = db.segment_watermarks(video_id)
842     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
843     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
844     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
845     # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
846     # they're dropped on the cloud path in this slice ? a documented follow-up.)
847     remote = _maybe_dispatch_remote(
848         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
849     )
850     if remote is not None:
851         response, segmentation_ran = (
852             remote # `ran` = did the cloud job actually execute (report)
853         )
854         return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
855     # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
856     compute_actual = "in_process"
857     segmenter = segmenter_cls(db, config)
858     try:
859         # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
860         # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
861         with _LOCAL_COMPUTE_LANE:
862             result = segmenter.process_video(
863                 video_id, local_video, req.player_pool, req.max_frames
864             )
865     except Exception:
866         # A raising segmenter must not leave half-written rows: restore the pre-run
867         # state (same destroy-after-confirm contract as the Failure branch), then
let
868         # the job worker record the failure.
869         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
870         raise
871     segmentation_ran = True
872
873     if isinstance(result, Failure):
874         logger.error(f"[API] Segmenter failed: {result.message}")
875         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
876         response = {
877             "video_id": video_id,
878             "status": "failed",
879             "message": f"Segmenter '{seg_name}' failed: {result.message}",
880             "results": [r.model_dump() for r in val_results],

```

```

881         "play_segments": [],
882     }
883     return response
884
885     segments = result.value if hasattr(result, "value") else result
886     notify("finalizing")
887     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
888
889     response = {
890         "video_id": video_id,
891         "status": "success",
892         "message": f"Successfully indexed {len(segments)} play segments using
893 '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
894         "results": [r.model_dump() for r in val_results],
895         "play_segments": segments,
896     }
897     return response
898 finally:
899     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
already set
900     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
other path stamp
901     # the actual path here so the job result PROVES which backend ran (in_process /
not_run) and
902     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
then visible.
903     if isinstance(response, dict):
904         response.setdefault(
905             "compute",
906             {
907                 "path": compute_actual or "not_run",
908                 "requested": getattr(req, "compute", None),
909             },
910         )
911     prov = response.get("compute", {}) if isinstance(response, dict) else {}
912     logger.info(
913         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
914         video_id,
915         getattr(req, "compute", None),
916         prov.get("path"),
917         prov.get("dispatched"),
918         (response or {}).get("status") if isinstance(response, dict) else None,
919     )
920     # Always emit the per-video observability report (next to the video, or to
report.output_dir). Never raises. Surface the paths in the response. (The
compute-path
921     # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
922     # doesn't surface arbitrary run signals.)
923     paths = _main.emit_indexer_report(
924         db=db,
925         video_id=video_id,
926         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
927         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
928         # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
929         # still preserved in DB provenance via db.add_video.
930         video_path=local_video,
931         config=_report_config_for_input(config, req.video_path),
932         val_results=val_results,
933         val_context=val_context,
934         segmenter_requested=req.segmenter,

```

```

935         segmenter_actual=segmenter_actual,
936         was_reingest=was_reingest,
937         segmentation_ran=segmentation_ran,
938     )
939     if paths and isinstance(response, dict):
940         response["reports"] = paths
941         storage.cleanup_acquired_local_input(
942             req.video_path, local_video, getattr(config, "storage", None)
943         )
944
945
946     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
947         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
948
949         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
950         that needs a missing cloud SDK, the network, or a credential returns ``None`` so the free-space
951         guard degrades to a no-op rather than blocking a job it cannot measure."""
952         try:
953             sc = (
954                 storage_config.model_dump()
955                 if hasattr(storage_config, "model_dump")
956                 else (storage_config or {})
957             )
958             backend = storage.get_storage(input_ref.backend, config=sc)
959             return int(backend.stat(input_ref).size)
960         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
961             not block
962             logger.debug(
963                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
964             )
965             return None
966
967     class _CompileError(Exception):
968         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
969         the API path maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
970         result."""
971
972         def __init__(self, status_code: int, detail: str):
973             super().__init__(detail)
974             self.status_code = status_code
975             self.detail = detail
976
977     def _resolve_compile(
978         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
979     ):
980         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
981         resolve, since a row could change between submit and run). Verifies the video exists, the requested
982         segments match and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
983         shot_count are exempt so manual/legacy picks can't silently vanish), and the output name is traversal-
984         safe. Raises ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
985         safe_out_name)``."""
986         video = db.get_video(video_id)
987         if not video:
988             raise _CompileError(404, "Indexed video record not found.")
989         # Reject path traversal / absolute names (os.path.join would otherwise write
990         anywhere on disk).

```



```

989         try:
990             out_name = safe_output_filename(output_filename)
991         except ValueError as e:
992             raise _CompileError(400, str(e)) from e
993
994     all_segments = db.query_play_segments(video_id, {})
995     matched = [s for s in all_segments if s["id"] in segment_ids]
996     if not matched:
997         raise _CompileError(400, "No matching play segments found to stitch.")
998
999     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1000     kept = []
1001     for s in matched:
1002         meta = s.get("metadata") or {}
1003         sc = meta.get("shot_count") if isinstance(meta, dict) else None
1004         try:
1005             if sc is not None and int(sc) < stitching.min_shot_count:
1006                 continue
1007             except (TypeError, ValueError):
1008                 pass # unparseable count -> treat as unknown, keep
1009             kept.append(s)
1010     if len(kept) < len(matched):
1011         logger.info(
1012             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
1013             floor.",
1014             stitching.min_shot_count,
1015             len(matched) - len(kept),
1016             len(matched),
1017         )
1018     if not kept:
1019         raise _CompileError(
1020             400,
1021             f"All {len(matched)} matching segments fall below "
1022             f"stitching.min_shot_count={stitching.min_shot_count}.",
1023         )
1024     return video, kept, out_name
1025
1026 def reel_object_uri(cfg, video_id: str, name: str) -> str:
1027     """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1028     (#463), or "" when
1029     the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
1030     compile
1031     mirror-upload and the /api/highlights redirect:
1032     ``<output_root>/highlights/<video_id>/<name>``. """
1033     root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1034     if not root or "://" not in root:
1035         return ""
1036     return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1037
1038 def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1039     """The active storage settings dict (per-backend config for the storage layer ? S3
1040     endpoint/region,
1041     GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
1042     default backend
1043     isn't silently constructed as a fresh default. """
1044     sc = getattr(cfg, "storage", None)
1045     return sc.model_dump() if sc is not None else None
1046
1047 def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1048     """A short-lived signed URL for a reel mirrored to the output bucket, so
1049     /api/highlights can
1050     307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the

```

```

ephemeral local
1047     /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1048     the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1049     reel_uri = reel_object_uri(cfg, video_id, name)
1050     if not reel_uri:
1051         return None
1052     settings = _storage_settings(cfg)
1053     if storage.exists(reel_uri, config=settings):
1054         return storage.signed_url(reel_uri, config=settings, method="GET")
1055     return None
1056
1057
1058     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1059         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1060         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1061         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1062         so the SPA always gets a clean verdict instead of a worker-crash error."""
1063         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1064
1065         notify = progress or (lambda stage: None)
1066         params = json.loads(job.get("params") or "{}")
1067         video_id = job.get("video_id") or ""
1068         segment_ids = params.get("segment_ids") or []
1069         locale = params.get("locale")
1070
1071         notify("resolving")
1072         try:
1073             video, kept, out_name = _resolve_compile(
1074                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
1075             )
1076         except _CompileError as e:
1077             return {"status": "failed", "message": e.detail, "video_id": video_id}
1078
1079         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1080         output_dir = (
1081             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1082         )
1083         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1084         # unknown/absent locale keeps the configured default unchanged.
1085         localized_watermark = localize_watermark(stitching.watermark, locale)
1086         compiler = _main.VideoCompiler(
1087             output_dir,
1088             begin_padding=stitching.begin_padding,
1089             end_padding=stitching.end_padding,
1090             watermark=localized_watermark,
1091         )
1092         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1093
1094         notify("stitching")
1095         local_src = ""
1096         try:
1097             # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1098             # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1099             # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1100             local_src = storage.acquire_local_input(
1101                 video["filepath"], getattr(cfg, "storage", None)

```

```

1102         )
1103         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1104         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1105         with _LOCAL_COMPUTE_LANE:
1106             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1107         except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1108             logger.error(
1109                 "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1110             )
1111             return {
1112                 "status": "failed",
1113                 "message": f"Highlight stitching failed: {e}",
1114                 "video_id": video_id,
1115             }
1116         finally:
1117             if local_src:
1118                 storage.cleanup_acquired_local_input(
1119                     video["filepath"], local_src, getattr(cfg, "storage", None)
1120                 )
1121
1122         # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1123         # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1124         # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1125         # still falls back to the local file on the same instance).
1126         reel_uri = reel_object_uri(cfg, video_id, out_name)
1127         if reel_uri:
1128             try:
1129                 notify("uploading reel")
1130                 # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1131                 # project/client) is used, not a fresh default.
1132                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1133             except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1134                 logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1135
1136         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1137         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1138         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1139         notify("finished")
1140         return {
1141             "status": "success",
1142             "filepath": out_path,
1143             "download_url": download_url,
1144             "video_id": video_id,
1145         }
1146

```

21. assistant (2026-07-07T15:16:31.336Z)

Now let me read the new guardrail tests in the test file.

22. user (2026-07-07T15:16:33.763Z)

```

700         fake = _FakeCloudClient(video_id="vidBoundary")
701         _inject_fake(monkeypatch, fake)
702         app = m.create_app(m.global_config, db)
703         client = TestClient(app)

```

```

704
705     r = client.post(
706         "/api/process",
707         json={
708             "video_path": "gs://b/clip.mp4",
709             "segmenter": "wasb_hybrid",
710             "compute": "cloud",
711         },
712     )
713     assert r.status_code == 202
714     job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
715     assert job["status"] == "done"
716     assert (
717         len(fake.calls) == 1
718     ) # 'cloud' honoured across the boundary ? dispatched despite local default
719     segs = db.query_play_segments(job["video_id"], {})
720     assert len(segs) == 2
721     assert all(_meta(s)["source"] == "cloud_run" for s in segs)
722
723
724     # --- hosted control-plane guardrail: compute='local' for a weights-backed detector
725     # A hosted cloud-serving control-plane bakes NO detector weights (only the worker Cloud
726     # them), so a forced compute='local' for wasb/tracknet/fusion can't run in-process
727     # there. It must fail
728     # FAST with an actionable "run in the cloud" message ? not fall through to the in-
729     # process segmenter
730     # that dies deep with a cryptic "WASB weights not found" (observed live: job f29d41d0?,
731     # 2026-07-07).
732
733     def test_local_on_cloud_server_without_weights_fails_fast(cloud_env, monkeypatch):
734         """compute='local' + a weights-backed detector on a cloud_run server with NO local
735         weights ?
736         a clear pre-dispatch failure (never dispatched, never the cryptic deep 'weights not
737         found')."""
738         _db, _bucket, _mp = cloud_env
739         monkeypatch.delenv("WASB_WEIGHTS", raising=False)
740         m.global_config.indexing.wasb.weights_path =
741         "/nonexistent/wasb_badminton_best.pth.tar"
742         fake = _FakeCloudClient()
743         _inject_fake(monkeypatch, fake)
744
745         out = m._execute_processing(
746             m.ProcessRequest(
747                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
748             )
749         )
750         assert out["status"] == "failed"
751         assert "can't run detection locally" in out["message"]
752         assert "Run in cloud" in out["message"]
753         assert "weights not found" not in out["message"] # NOT the cryptic deep failure
754         assert fake.calls == [] # never dispatched
755         # Honest provenance: requested local, ran NOWHERE (pre-dispatch reject).
756         assert out["compute"]["requested"] == "local"
757         assert out["compute"]["dispatched"] is False
758         assert out["compute"]["path"] == "not_run"
759
760     def test_local_on_cloud_server_with_weights_present_runs_in_process(
761         cloud_env, tmp_path, monkeypatch
762     ):
763         """Escape hatch: the guardrail must NOT over-block ? on a cloud-wired box that
764         ACTUALLY has the

```

```

760     detector weights on disk, compute='local' still runs the in-process segmenter (no
dispatch)."""
761     _db, _bucket, _mp = cloud_env
762     weights = tmp_path / "wasb_badminton_best.pth.tar"
763     weights.write_bytes(b"WEIGHTS")
764     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
765     m.global_config.indexing.wasb.weights_path = str(weights)
766     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
767     fake = _FakeCloudClient()
768     _inject_fake(monkeypatch, fake)
769
770     out = m._execute_processing(
771         m.ProcessRequest(
772             video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="local"
773         )
774     )
775     assert out["status"] == "success"
776     assert fake.calls == [] # in-process, not dispatched
777     assert out["compute"]["path"] == "in_process"
778
779
780 def test_local_non_weights_segmenter_on_cloud_server_still_runs_in_process(
781     cloud_env, monkeypatch
782 ):
783     """The guardrail is scoped to weights-backed detectors: a weight-free segmenter
(motion) with
784     compute='local' on a cloud server is unaffected ? it runs in-process, never fails
fast."""
785     _db, _bucket, _mp = cloud_env
786     m.global_config.indexing.wasb.weights_path =
"/nonexistent/should_not_matter.pth.tar"
787     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)
788     fake = _FakeCloudClient()
789     _inject_fake(monkeypatch, fake)
790
791     out = m._execute_processing(
792         m.ProcessRequest(
793             video_path="gs://b/clip.mp4", segmenter="motion", compute="local"
794         )
795     )
796     assert out["status"] == "success"
797     assert fake.calls == []
798     assert out["compute"]["path"] == "in_process"
799
800
801 def test_local_weights_backed_on_pure_local_server_is_not_guardrailed(
802     tmp_path, monkeypatch
803 ):
804     """The guardrail only fires on a cloud-capable server. On a pure-local server (no
cloud), a
805     weights-backed compute='local' request falls through to the in-process segmenter as
before ? the
806     'run in the cloud' message would be nonsense there (no cloud to fall back to)."""
807     db = Database(str(tmp_path / "t.db")) # noqa: F841
808     monkeypatch.setattr(m, "db_instance", db)
809     cfg = AppConfig.model_validate({"deployment": {"compute_target": ""}}) # noqa: F841
810     monkeypatch.setattr(m, "global_config", cfg)
811     monkeypatch.setattr(
812         m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {})
813     )
814     monkeypatch.delenv("CLOUD_RUN_JOB", raising=False)
815     monkeypatch.delenv("GOOGLE_CLOUD_PROJECT", raising=False)
816     monkeypatch.delenv("WASB_WEIGHTS", raising=False)
817     cfg.indexing.wasb.weights_path = "/nonexistent/wasb.pth.tar"
818     monkeypatch.setattr(m.segmenter_registry, "get", lambda name: _InProcSeg)

```

```

819     video = tmp_path / "v.mp4"
820     video.write_bytes(b"hashable-bytes")
821
822     out = m._execute_processing(
823         m.ProcessRequest(
824             video_path=str(video), segmenter="wasb_hybrid", compute="local"
825         )
826     )
827     # Not the hosted fast-fail ? it ran in-process (the local box owns its own weights
story).
828     assert out["status"] == "success"
829     assert out["compute"]["path"] == "in_process"
830
831
832     # --- compute-path PROVENANCE (the validation method: prove which backend ran)
-----
833
834
835     def test_cloud_dispatch_stamps_compute_provenance(cloud_env):
836         """A cloud run stamps result['compute'] with path=cloud_run + the REAL Cloud Run
execution name
837         (the GCP-verifiable proof) + the requested choice + job/region/outputs ? so the
intended path is
838         provable from the job result alone (no guessing from segment metadata)."""
839         _db, _bucket, monkeypatch = cloud_env
840         _inject_fake(monkeypatch, _FakeCloudClient())
841
842         out = m._execute_processing(
843             m.ProcessRequest(
844                 video_path="gs://b/clip.mp4", segmenter="wasb_hybrid", compute="cloud"
845             )
846         )
847         prov = out["compute"]
848         assert prov["path"] == "cloud_run"
849         assert prov["requested"] == "cloud"
850         assert (
851             prov["execution"] == "exec-1"
852         ) # the fake client's execution name (real one in prod)
853         assert prov["outputs_uri"].endswith("/outputs/vidCloud/")
854         assert prov["job"] and prov["region"] # the dispatch coordinates are recorded
855
856
857     def test_local_override_stamps_in_process_provenance(cloud_env):
858         """On a cloud server, compute='local' runs in-process AND the result proves it:
path=in_process,
859         requested=local ? so a 'ran local despite a cloud server' is auditable (no silent
ambiguity)."""

```

23. assistant (2026-07-07T15:16:45.250Z)

The four new guardrail tests indeed only use `wasb\_hybrid` (3x) and `motion` (1x). Let me verify no OTHER test anywhere exercises the tracknet/fusion branches of `\_local\_detector\_weights\_present`.

24. user (2026-07-07T15:16:47.896Z)

C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_ablation.py:14: reproduces the hand-measured Gate-A result for either the legacy n=6 substrate or the refreshed C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_ablation.py:15: n=15 A7 substrate. Skips cleanly when the golden JSONs are absent (CI), runs when present C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_ablation.py:552: The legacy n=6 owner-box substrate preserves the 2026-06-14 hand measurement. After A7, the C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_ablation.py:553: refreshed owner-box substrate has n=15 full-fusion files; that result is intentionally weaker C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_ablation.py:577:

```

# The A7 refreshed n=15 substrate: proposal-side structural signal, but CI crosses zero.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_config.py:60:
"""The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no mask.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_config.py:69:
assert f.substrate == "wasb"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_config.py:92:
to fusion_hybrid was REVERTED as a confirmed served regression (eval?serve)."""
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_config.py:122:
"substrate": "tracknet",
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_config.py:133:
assert f.substrate == "tracknet"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_config.py:257:
assert cfg.indexing.tracknet.velocity_threshold == 0.02
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_config_wasb.py:5:shuttle substrate's config) was discarded ? config.json
overrides did nothing.
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_eval_harness.py:39: db.add_candidate_segment("vid1", "tracknet_hybrid",
40.0, 50.0)
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:5:substrate windows; served Gate A drops sub-threshold
windows from the AI handoff while
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:13:from backend.pipeline.segmenters.fusion_hybrid import
FusionSegmenter
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:115:def
test_persisted_candidates_are_superset_of_substrate_windows():
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:124: ] # exactly the substrate windows, unfiltered
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:134: # Persistence is still the full superset (offline
substrate intact)...
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:210:def test_build_runner_selects_substrate():
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:211: """_build_runner picks the WASB or TrackNet runner
by indexing.fusion.substrate."""
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:213: with
patch("backend.pipeline.segmenters.fusion_hybrid.WasbRunner") as W, patch(
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:214:
"backend.pipeline.segmenters.fusion_hybrid.WasbConfig"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:216:
"backend.pipeline.segmenters.fusion_hybrid.TrackNetRunner"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:217: ) as T,
patch("backend.pipeline.segmenters.fusion_hybrid.TrackNetConfig") as TC:
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:220: _runner_w, extras_w =
seg._build_runner({"fusion": {"substrate": "wasb"}})
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:221: _runner_t, extras_t =
seg._build_runner({"fusion": {"substrate": "tracknet"}})
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:223: assert W.called and extras_w["fusion_substrate"] ==
"wasb"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:224: assert T.called and extras_t["fusion_substrate"] ==
"tracknet"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:226: assert extras_d["fusion_substrate"] == "wasb"
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\tests\test_fusion_hybrid.py:226:

```

```

indexer\tests\test_job_self_scheduling.py:1: """Tests for the v3 self-scheduling job substrate
(EXECUTION_POLICY.md P2).
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_native_wasb_runner.py:594:     from backend.pipeline.segmenters.fusion_hybrid
import FusionSegmenter
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_native_wasb_runner.py:600:     assert params.get("fusion_substrate") == "wasb"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_native_wasb_runner.py:605:     from backend.pipeline.segmenters.fusion_hybrid
import FusionSegmenter
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_native_wasb_runner.py:609:         {"detector_impl": "native", "fusion":
{"substrate": "tracknet"}}
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_native_wasb_runner.py:615:     from
backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_personalization_serving.py:18:         "substrate": "wasb",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_personalization_serving.py:79:     assert out["substrate"] == "wasb"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_served_gate_a_regression.py:3:The flagged gap (#146): the served default flip
(``default_segmenter=fusion_hybrid`` +
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_served_gate_a_regression.py:4:``min_crossings=2``) is **mock-tested only** on
real video (``test_fusion_hybrid.py`` drives a
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_tracknet_hybrid.py:7:from backend.pipeline.segmenters.tracknet_hybrid import
TrackNetHybridSegmenter
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_tracknet_hybrid.py:31:     fn =
patch("backend.pipeline.segmenters.tracknet_hybrid.TrackNetRunner")(fn)
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_tracknet_hybrid.py:89:     assert kwargs["detector"] == "tracknet_hybrid"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_tracknet_hybrid.py:348:     assert seg.name == "tracknet_hybrid"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_trajectory_hybrid_equivalence.py:3:tracknet_hybrid and wasb_hybrid were copy-
paste twins that drifted (the
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_trajectory_hybrid_equivalence.py:14:from
backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_trajectory_hybrid_equivalence.py:20:     (TrackNetHybridSegmenter, "tracknet",
"tracknet_hybrid"),
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_trajectory_hybrid_equivalence.py:98:     the fallback while tracknet_hybrid
short-circuited. One implementation now."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-
indexer\tests\test_trajectory_hybrid_equivalence.py:193:     assert "tracknet_hybrid" in
available
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_windowing.py:59:
"""The eval substrate / golden JSONs must stay valid ? presets unchanged in value."""
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py:779:
"fusion_hybrid",
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py:782:
assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"
C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test_worker.py:888:
"tracknet_hybrid",

```

25. user (2026-07-07T15:16:50.572Z)

No matches found

26. assistant (2026-07-07T15:18:19.346Z)

The new test file has zero matches for tracknet/fusion ? confirming those branches are untested by the guardrail tests. The other hits are unrelated tests (config parsing, segmenter runner selection, worker CLI). Now I must assess materiality: does the branchy logic read real, divergent config fields? Let me verify the config model and whether the finding's scenario holds against the real deployment premise.

27. user (2026-07-07T15:18:22.281Z)

```
46:class TrackNetConfig(BaseModel):
51:    weights_path: str = ""
68:class WasbIndexConfig(BaseModel):
69:    """Typed config for the live WASB shuttle substrate (indexing.wasb).
80:    weights_path: str = (
168:class FusionConfig(BaseModel):
179:    # Which trajectory substrate seeds the windows (shuttle stays primary).
180:    substrate: Literal["wasb", "tracknet"] = "wasb"
182:    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
202:class IndexingConfig(BaseModel):
293:    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294:    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295:    fusion: FusionConfig = Field(default_factory=FusionConfig)
```

28. user (2026-07-07T15:18:25.422Z)

```
46    class TrackNetConfig(BaseModel):
47        wsl_distro: str = "Ubuntu"
48        repo_dir: str = "~/models/TrackNetV4"
49        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50        conda_env: str = "TrackNetV4"
51        weights_path: str = ""
52        queue_length: int = 5
53        stage_in_wsl: bool = True
54        wsl_stage_dir: str = "~/clips"
55        timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56        velocity_threshold: float = 0.02
57        # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58        # decoded frame cache) are deleted automatically ? they are pure intermediates,
59        # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60        # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61        keep_frames: bool = False
62        # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63        # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64        # 0 = no extra floor (the headroom guard still applies).
65        min_free_gb: float = 0.0
66
67
68    class WasbIndexConfig(BaseModel):
69        """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71        Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72        these knobs actually take effect ? previously this whole block was silently dropped
73        because nested config sections defaulted to extra='ignore'.
74        """
75
76        wsl_distro: str = "Ubuntu"
77        repo_dir: str = "~/models/WASB-SBDT"
78        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79        conda_env: str = "wasb"
80        weights_path: str = (
81            "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82        )
83        sport: str = "badminton"
84        wsl_stage_dir: str = "~/clips"
```

```

85         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86         velocity_threshold: float = 0.02
87         # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88         ---
89         # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
90         # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
91         # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
92         device: str = "cuda" # auto | cuda | mps | cpu
93         python_bin: str = (
94             "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
95         )
96         # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache

```

29. assistant (2026-07-07T15:19:15.360Z)

Now I have the concrete config facts:

```

- `TrackNetConfig.weights_path` defaults to ``""`` (empty)
- `WasbIndexConfig.weights_path` defaults to `"/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"`
- `FusionConfig.substrate` defaults to `"wasb"`

```

Let me concretely execute the actual function logic to (a) confirm the current code routes correctly with no present bug, and (b) prove a mutation to the tracknet branch survives all four new tests. This is the crux of whether the "failure" occurs against real code as written vs. only after a hypothetical refactor.

30. user (2026-07-07T15:19:40.399Z)

=== The FOUR NEW TESTS only touch these two seg names ===

```

T1 wasb_hybrid absent : CUR=False MUT=False (test asserts False/fail-fast)
T2 wasb_hybrid present: CUR=True MUT=True (test asserts True/in_process)
T3 motion              : CUR=True MUT=True (test asserts in_process)

```

=== The UNTESTED branches (tracknet / fusion+tracknet) ? does MUTATION diverge? ===

```

tracknet_hybrid tn-present/wasb-absent : CUR=True MUT=False <-- correct=True; MUT wrongly
blocks
tracknet_hybrid tn-absent/wasb-present : CUR=False MUT=True <-- correct=False; MUT wrongly
escapes->dies deep
fusion+tracknet tn-absent/wasb-present : CUR=False MUT=True <-- correct=False; MUT wrongly
escapes

```

=== Canonical hosted control-plane: NO weights on disk at all (the fix's premise) ===

```

fusion+tracknet, default paths, nothing on disk: CUR=False MUT=False <-- both False => both
fail-fast

```

31. assistant (2026-07-07T15:20:59.958Z)

The execution is decisive. Let me capture the key evidence precisely.

The results confirm three things against the real code:

1. ****The four new tests produce byte-identical results under current and mutated code**** (T1: False/False, T2: True/True, T3: True/True). A routing regression that drops the substrate switch ships green ? the coverage gap is real and mutation-confirmed.
2. ****The current code routes correctly for every input**** (tracknet tn-present?True, tn-absent?False; fusion+tracknet?reads tracknet; fusion default?reads wasb). There is no wrong result against the code as written.
3. ****On the canonical hosted control-plane (no weights on disk ? the fix's own premise), even the mutated routing returns False ? still fails-fast.**** The finding's dramatic "silently dies deep with a green suite" outcome does not occur on the targeted deployment; it needs an atypical

partial-weights box (WASB present on disk, TrackNet absent).

One more correction to verify: whether the `else`/wasb branch is actually already covered by the wasb_hybrid tests (making "the entire branchy half" an overstatement).

32. user (2026-07-07T15:21:01.931Z)

```
179         if name == "tracknet_hybrid" or (name == "fusion_hybrid" and substrate ==
"tracknet"):
180             path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""
181         else: # wasb_hybrid, or fusion on the (default) WASB substrate
182             path = (
183                 os.environ.get("WASB_WEIGHTS")
184                 or getattr(getattr(idx, "wasb", None), "weights_path", "")
185                 or ""
186             )
187         return bool(path) and os.path.exists(os.path.expanduser(path))
```

33. assistant (2026-07-07T15:21:24.926Z)

Confirmed: line 180 (the `tracknet` path) is the only genuinely uncovered line. The `else` branch (181-186, the WASB path) is exactly what the three `wasb\_hybrid` guardrail tests drive, and `fusion\_hybrid` on its default `wasb` substrate takes that same already-covered branch. I have enough to rule.

34. user (2026-07-07T15:21:38.702Z)

Structured output provided successfully